

SOP-Guard

Agentic RAG for Procedurally-Faithful Clinical Decision Support

Research Proposal & System Design Document

Project: ClinicalSOP-RAG with Procedural Faithfulness Verification
Focus: Hospital SOP Question-Answering with Agentic Verification
Date: June 2026

Table of Contents

1. System Overview
2. Key Advanced Features
3. System Architecture
4. Technology Stack
5. Datasets
6. Voice Interface Analysis
7. Why This System Will Be Used in Hospitals
8. Thesis Summary

1. System Overview

SOP-Guard is an AI system that helps doctors and nurses get instant, accurate answers to questions about hospital Standard Operating Procedures (SOPs). When a clinician asks something like *"What's the sepsis protocol for a patient with renal impairment?"*, the system retrieves the relevant SOP sections, generates a grounded answer, and then **automatically verifies** that the answer doesn't violate any clinical thresholds, skip required steps, or miss contraindications.

What makes SOP-Guard advanced is its **agentic architecture**: instead of a single LLM call, multiple specialized AI agents collaborate — one retrieves evidence, one reasons about the answer, one verifies procedural correctness, and one explains the reasoning chain to the clinician. The system learns from clinician feedback over time, flags its own uncertainty, and is designed to run on hospital infrastructure without sending patient data to external servers.

2. Key Advanced Features

Agentic Pipeline (LangGraph-based)

- Multi-agent orchestration with conditional routing — not a fixed chain
- Agents can loop back (e.g., verifier requests additional retrieval if evidence is insufficient)

Procedural Faithfulness Verifier (Core Research Contribution)

- Checks three violation types: threshold errors, sequence violations, contraindication misses
- Uses structured SOP representations (not just raw text) for formal verification
- Produces a faithfulness score with specific violation citations

Confidence-Gated Output

- Every response carries a calibrated confidence score
- Low-confidence answers trigger automatic escalation with links to source SOP
- Abstention is preferred over hallucination

Explainability Layer

- Step-by-step reasoning trace: which SOP sections retrieved → how answer composed → what verifier checked
- Highlighted source passages with relevance scores
- Counterfactual explanations: "If the patient had [X], the answer would change because..."

Clinician Feedback Loop

- Thumbs up/down + optional free-text correction on every response
- Corrections used to: flag SOPs needing updates, fine-tune retrieval ranking, add hard-negative examples
- Monthly feedback digest for hospital quality teams

Memory System

- Short-term: Conversation context within a session (multi-turn clarification)
- Long-term: Per-department query patterns — surfaces FAQs, identifies confusing SOP sections

Structured SOP Knowledge Base

- SOPs parsed into semi-structured format: steps → preconditions → decision points → thresholds → contraindications
- Enables formal rule-checking beyond what pure LLM reasoning can do

3. System Architecture

Stage 1: Query Intake

Clinician submits a question via web/mobile interface. The **Query Understanding Agent** classifies the query type (threshold check, sequence question, contraindication lookup, general SOP question) and extracts key clinical entities (drug names, conditions, patient parameters).

Stage 2: Retrieval

The **Retrieval Agent** performs hybrid search over the SOP knowledge base: dense retrieval (embedding similarity) for semantic matching, sparse retrieval (BM25) for exact term matching (drug names, dosage numbers), and metadata filtering by department, SOP category, and recency. A cross-encoder re-ranks to select top-k passages. If evidence is insufficient, the agent reformulates the query and retries once.

Stage 3: Answer Generation

The **Reasoning Agent** synthesizes an answer grounded strictly in retrieved passages using structured prompting that forces citation of source passages. For multi-step procedures, it outputs a numbered step list with decision branch points.

Stage 4: Procedural Verification

The **Verifier Agent** checks the generated answer against the structured SOP representation: **Threshold check** — Are all numerical values consistent with the SOP? **Sequence check** — Are steps in the correct order? Are any mandatory steps omitted? **Contraindication check** — Does the answer account for stated contraindications? On FAIL: the answer is regenerated with the violation flagged as a constraint (max 1 retry, then escalation).

Stage 5: Confidence Scoring & Output

Confidence score computed from: retrieval relevance + verifier result + answer-source overlap. Three output tiers: **High confidence** (full answer displayed), **Medium confidence** (answer with advisory banner + link to source SOP), **Low confidence** (answer withheld, clinician directed to source SOP or human expert).

Stage 6: Feedback Collection

Response displayed with source highlights, reasoning trace, and feedback buttons. Feedback stored in a structured log with query, response, clinician role, department, and correction text.

LangGraph Orchestration Flow

```
QueryIntake → Retrieval → [sufficient?] → Reasoning → Verification → [pass?] →  
ConfidenceGate → Output → Feedback
```

The graph has conditional edges: the verifier can route back to retrieval or reasoning, and the confidence gate can route to escalation. This is a core advantage of LangGraph over linear chains.

4. Technology Stack

Component	Technology
LLM (Generation)	Claude Sonnet 4.6 (primary); Llama 3.1 70B (local fallback)
LLM (Verification)	Same LLM with structured JSON output for formal checks
Orchestration	LangGraph (deterministic graph with conditional routing)
Embeddings	text-embedding-3-large or BGE-M3 (open-source)
Vector Database	Qdrant (hybrid search, metadata filtering, on-prem)
Sparse Retrieval	BM25 via Elasticsearch or Qdrant sparse vectors
Cross-Encoder	ms-marco-MiniLM-L12 or Cohere Rerank
SOP Parsing	PyMuPDF + LLM-assisted structuring into JSON
Backend	FastAPI (Python)
Frontend	React + Tailwind CSS (mobile-responsive PWA)
Feedback Store	PostgreSQL (structured logs)
Deployment	Docker / Kubernetes for hospital data center
Evaluation	RAGAS + custom faithfulness metrics

Why LangGraph Over Alternatives

- **vs. simple chains:** LangGraph supports cycles (verifier → re-retrieval) and conditional branching, essential for the verify-and-retry loop.
- **vs. CrewAI/AutoGen:** Those frameworks are designed for autonomous multi-agent chat, which introduces unpredictability. SOP-Guard needs a deterministic, auditable control flow.
- **vs. DSPy:** DSPy is strong for prompt optimization but doesn't natively support stateful agent orchestration.

5. Datasets

5.1 Public Datasets for Training & Evaluation

Dataset	Use	Access
PubMedQA	Clinical QA evaluation; 1,000 expert-labeled pairs	Free (HuggingFace)
MedQA (USMLE)	Clinical reasoning; 12,723 MCQs	Free (GitHub)
MedMCQA	194K medical MCQs covering clinical procedures	Free (HuggingFace)
MIMIC-IV (Notes)	Procedure descriptions for entity extraction	PhysioNet (credentialed)
WikiDoc / StatPearls	Clinical reference articles for retrieval testing	Free
ExpertQA	2,177 expert-validated QA pairs incl. medicine	Free (GitHub)
MTSamples	Medical transcription samples for entity extraction	Free

5.2 Creating the SOP Dataset

No public SOP dataset exists. The dataset must be constructed through the following steps:

1. **Collect real SOPs:** Partner with 1–2 hospitals. Start with 3–5 high-impact departments (Emergency, ICU, Pharmacy, Infection Control, Blood Bank). Target 50–100 SOPs.
2. **Augment with public clinical guidelines:** WHO clinical guidelines, CDC infection control guidelines, AHA/ACLS resuscitation protocols, NICE guidelines (UK), Surviving Sepsis Campaign guidelines.
3. **Structure each SOP into JSON schema:** Fields include sop_id, title, department, ordered steps (with preconditions, time constraints, contraindications), thresholds (parameter, operator, value, unit, action).
4. **Generate QA pairs:** For each SOP, use an LLM to generate 20–50 question-answer pairs, then have a clinical expert validate them. Target 2,000+ validated QA pairs.
5. **Create adversarial test cases:** Deliberately craft questions designed to trigger each violation type (wrong threshold, skipped step, missed contraindication). These form the procedural faithfulness evaluation set.

5.3 Evaluation Datasets & Metrics

What to Evaluate	Dataset / Method	Metric
Retrieval quality	SOP QA pairs with gold passages	Recall@5, MRR
Answer correctness	Expert-labeled QA set	Accuracy, F1
Procedural faithfulness	Adversarial violation set (novel)	Violation detection P/R
Hallucination detection	Answer-source contradiction pairs	Faithfulness (RAGAS)

Clinical safety	Expert review of 200 random outputs	Safety pass rate
-----------------	-------------------------------------	------------------

6. Voice Interface Analysis

Decision: Exclude from v1

Arguments Against (for v1):

- Hospital environments are noisy (alarms, conversations, PA systems) — ASR error rates in ICUs are 15–30% higher than quiet settings.
- Clinical terminology misrecognition is dangerous ("heparin" vs "Hespan").
- Privacy concerns with speaking patient-adjacent queries aloud.
- Regulatory complexity.

Arguments For:

- Hands-free use during procedures.
- Faster than typing during emergencies.

Recommendation: Design the API to be voice-ready (text-in, text-out with structured responses), but ship v1 as text-only. Voice can be added as a v2 feature after the core system is validated, and only for non-bedside use cases (e.g., nursing station queries).

7. Why This System Will Actually Be Used in Hospitals

It solves a real, daily problem. Nurses and junior doctors look up SOPs multiple times per shift. Current method: dig through a PDF repository, Ctrl+F, read 20 pages. SOP-Guard reduces this to a 10-second interaction.

It doesn't require EHR integration. This is the single biggest barrier to hospital AI adoption. By working only with SOP documents (not patient data), SOP-Guard avoids HIPAA/data-sharing complications and can be deployed in weeks, not years.

It can run on-prem. Using an open-source LLM (Llama 3.1), the entire system runs inside the hospital network. No patient data, no SOP content leaves the building.

It builds trust through transparency. Every answer shows its sources, its reasoning, and its confidence level. Clinicians can verify in seconds. The system explicitly says "I'm not sure" rather than guessing.

It improves over time. Clinician feedback identifies which SOPs are confusing, which questions recur, and where the system fails. This creates value for hospital quality improvement teams.

Incremental deployment path: Start as a read-only reference tool in one department. Expand based on usage data and feedback. No workflow disruption — it's additive, not replacement.

8. Thesis Summary

What Problem Are We Solving?

Hospitals run on Standard Operating Procedures — step-by-step instructions for everything from managing a sepsis patient to handling a blood transfusion reaction. There are hundreds of these documents, they're long, and they get updated frequently. When a nurse needs to check a dosage threshold at 3 AM, they're searching through PDF files under time pressure. Mistakes happen — not because clinicians are careless, but because the information access system is poor.

What Are We Building?

We're building an AI assistant specifically designed to answer questions about hospital SOPs. You ask it a question in plain language — *"What's the maximum heparin dose for a patient with renal impairment?"* — and it gives you a direct answer with the exact source highlighted.

But here's what makes this system different from just putting ChatGPT in a hospital: **it checks its own work**. After generating an answer, a separate AI component verifies that the answer doesn't contain the three most dangerous types of errors in clinical procedures: wrong numbers (dosage thresholds, time windows), skipped steps (missing a required action in a sequence), and missed contraindications (failing to account for conditions that change the protocol).

Why Is This Research, Not Just Engineering?

The core research contribution is the **Procedural Faithfulness Verifier**. Current AI systems can retrieve relevant documents and generate fluent answers, but they have no mechanism to check whether the answer is *procedurally correct* — whether the steps are in the right order, whether the thresholds are accurate, whether exceptions are properly handled. We formalize this as a verification problem, build a structured representation of SOPs that enables automated checking, and evaluate how well the system detects violations compared to human experts.

We also contribute an **agentic architecture** where multiple AI components collaborate with explicit control flow — the system can recognize when it doesn't have enough evidence, go back and search again, and ultimately refuse to answer rather than risk giving wrong information. This "verify-then-respond" pattern is a design principle we argue should be standard for any AI system used in safety-critical settings.

What's the Expected Outcome?

We expect to demonstrate that the verification layer catches 85–95% of procedural errors that a standard RAG system would miss, with minimal impact on response time (under 5 seconds total). We'll publish the verification framework, the structured SOP schema, and the adversarial evaluation dataset as open resources for the clinical AI community.

The system is designed to be deployed in a real hospital setting — we deliberately avoided requiring EHR integration, chose technologies that run on local servers, and built the interface for the reality of clinical work (time pressure, mobile devices, interruptions). The goal is a system that a hospital IT team can install in a week and clinicians will actually use.